Domain Name System (DNS)

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Explain what DNS is and why it is foundational to the Internet and cloud infrastructure
- Describe the DNS hierarchy: root servers, TLDs, authoritative name servers, and resolvers
- Distinguish between recursive and iterative DNS resolution
- Read and interpret common DNS resource record types (A, AAAA, CNAME, MX, NS, TXT, SOA, CAA)
- Explain DNS zone delegation and how authority is transferred between zones
- Describe how cloud DNS services (AWS Route 53, Google Cloud DNS) map to DNS concepts
- Configure DNS records and understand TTL trade-offs in a cloud context
- Recognize DNS-related failure modes and common troubleshooting patterns

What is DNS & Why It Matters

The Core Problem DNS Solves

- Computers communicate using IP addresses (e.g., 216.58.219.196)
- Humans communicate using names (e.g., google.com)
- DNS is the translation layer between the two — often called **the phonebook of the Internet**
- Without DNS, every user would need to memorize IP addresses to visit any website

What DNS Is

- A **globally distributed, hierarchical, scalable database** that maps names to IP addresses
- Not a single server — a coordinated system of millions of servers worldwide
- No single point of control or failure

Key Properties of DNS

Property	What It Means
Distributed	Data is maintained locally but retrievable globally; no single server has all data
Scalable	No limit on database size or number of queries; queries spread across many servers
Reliable	Data is replicated; clients can query primary, secondary, or cached data
Dynamic	Master database can be updated; changes replicate to secondary servers
Loosely Coherent	Changes propagate over time (not instantly); TTL controls caching behavior

DNS Architecture & System Properties

Global Distribution Model

- Data is **maintained locally** by whoever owns the domain
- Data is **cached globally** to improve performance
- Remote DNS data is locally cacheable — this is why changes take time to propagate

```
Your Browser
     |
Local Resolver (your ISP or 8.8.8.8)
     |
Root Server (13 logical root servers, globally mirrored)
     |
TLD Server (.com, .org, .edu, .io, ...)
     |
Authoritative Name Server (knows YOUR domain)
     |
Answer: IP address returned
```

Loose Coherency and TTL

- Each **zone** (a portion of the DNS namespace) has a **serial number**
- Serial number increments on every change to that zone
- Secondary servers periodically check: "Is your serial number higher than mine? Send me the new data."
- **TTL (Time to Live)** — every record has a TTL value in seconds
  - High TTL (e.g., 86400 = 24 hours): faster lookups, slow to propagate changes
  - Low TTL (e.g., 60 seconds): slower lookups, changes propagate fast
  - **Cloud tip:** Before migrating or changing infrastructure, reduce TTL to 60s a day in advance, then change, then raise TTL back

The Three Components of DNS

DNS is built from three interacting components:

1. **Name Space** — the hierarchical tree structure of domain names
2. **Name Servers** — servers that store and respond to queries about the name space
3. **Resolvers** — clients that query name servers on behalf of applications

The DNS Namespace & Hierarchy

The Inverted Tree

DNS is structured as an **inverted tree**, with the root at the top:

- The root is represented by a single dot: .
- Reading a fully qualified domain name (FQDN) **right to left** traces the path from root to leaf
- Example: www.google.com. → root (.) → .com → google → www

```
                . (root)
               /|\
              / | \
          .com .org .edu  .gov  .uk  .io ...
          /  \
     google   amazon
     /   \
  www   mail
```

Labels

- Each node in the tree has a **label** — a string of up to 63 bytes
- Valid characters: A–Z, a–z, 0–9, and - (hyphen)
- Labels are **case-insensitive**: Google.com and google.com are the same
- Sibling nodes (nodes at the same level with the same parent) must have **unique labels**
- The null label (empty string) is reserved for the **root node** only

Domain Names

A **domain name** is the full path from a node to the root, read left to right with dots as separators:

- Maximum length: **255 characters**
- Maximum depth: **127 levels**

```
  www   .   google   .   com   .
   |          |         |     |
 label      label       TLD   root (implied)
```

Top-Level Domains (TLDs)

- **Generic TLDs (gTLDs):** .com, .net, .org, .gov, .edu, .io
- **Country Code TLDs (ccTLDs):** .uk, .de, .cn, .us, .ca
  - All ccTLDs are exactly **two letters** (ISO 3166-1 alpha-2 country codes)
  - Currently **312 ccTLDs** in active use
  - **Risk with ccTLDs:** they are tied to a country's jurisdiction — e.g., after Brexit, UK entities lost eligibility for .eu domains

Subdomains and FQDNs

- Subdomains are domains within a larger domain
  - west.example.com and east.example.com are subdomains of example.com
- **Fully Qualified Domain Name (FQDN):** an absolute name specifying a node's exact location in the DNS tree
  - Technically ends with a dot: mail.google.com.
  - Most tools omit the trailing dot, but it is implied

Domain Registration: Who Controls What

- **ICANN** — nonprofit that manages the IP namespace, controls root servers, oversees TLD policy
- **Registry** — manages a specific TLD (e.g., Verisign manages .com)
- **Registrar** — sells domain registrations to end users (accredited by ICANN)
- **Registrant** — the person or organization that owns a domain

```
  ICANN (Internet Corporation for Assigned Names and Numbers)
    |
    |-- Accredits --> Registrars (GoDaddy, Namecheap, Google
Domains...)
                          |
                          |-- Register domains for --> Registrants
(you, your company)
```

Name Servers, Zones & Zone Delegation

Name Servers

Name servers are the servers that actually hold DNS data and respond to queries.

- Run DNS server software (e.g., **BIND**, **NSD**, **PowerDNS**, or cloud-managed)
- Store DNS data in units called **zones**
- Usually **multiple name servers** are authoritative for the same zone (redundancy)
- A single name server can be authoritative for **many different zones**

Types of Name Servers

- **Authoritative:** The source of truth for a zone. When you set DNS records in Route 53 or Google Cloud DNS, those are authoritative name servers.
- **Primary:** Holds the editable master copy of zone data
- **Secondary:** Holds read-only replicated copies; serves data if primary is unreachable
- **Caching/Recursive Resolver:** Your ISP's DNS server, or public resolvers like 8.8.8.8 (Google) or 1.1.1.1 (Cloudflare) — they do the work of resolution on your behalf

```
Name Servers
│
├── Authoritative (owns the data)
│   ├── Primary (master) — where zone data is edited/created
│   └── Secondary (slave) — receives replicated data from primary
│
└── Caching / Recursive Resolver (does not own data)
    └── Queries authoritative servers, caches results for TTL
duration
```

Zones

A **DNS zone** is a portion of the DNS namespace under the administrative control of a specific entity.

- A zone always starts with an **SOA record** (Start of Authority) and **NS records**
- One zone can span multiple subdomains — OR a subdomain can be delegated to its own zone

```
example.com zone (managed by you)

example.com          (SOA, NS)
www.example.com      (A record)
api.example.com      (A record)
mail.example.com     (MX record)
```

Zone Delegation

Zone delegation is the process by which a parent zone hands off authority over a subdomain to a different set of name servers. This is how the entire DNS hierarchy works — delegation all the way down from the root.

How delegation works step by step:

```
  Root Zone (.)
    └── delegates .com to Verisign's name servers
            └── .com zone delegates example.com to YOUR name servers
                    └── example.com zone can delegate
                        dev.example.com to a DIFFERENT set of name
servers
```

Concrete example — delegating dev.example.com to its own zone

```
  In the PARENT zone (example.com zone file):

  ; NS records for the delegated child zone
  dev.example.com.   IN  NS  ns1.devteam.example.com.
  dev.example.com.   IN  NS  ns2.devteam.example.com.

  ; Glue records — needed because ns1/ns2 are INSIDE the delegated
zone
  ns1.devteam.example.com.  IN  A  203.0.113.10
  ns2.devteam.example.com.  IN  A  203.0.113.11
```

What are glue records?

- Glue records solve a circular dependency.
- If ns1.dev.example.com is the name server for dev.example.com, then to look up dev.example.com you need to ask ns1.dev.example.com — but you cannot find that address without first resolving dev.example.com.
- The glue record breaks this cycle by providing the IP address of the name server directly in the parent zone.

Why zone delegation matters in cloud?

- Large organizations delegate DNS zones to different teams (e.g., prod vs. dev environments)
- In AWS, you can create a **hosted zone** per environment and delegate from your root domain
- In GCP, Cloud DNS supports the same pattern with **public and private zones**
- Enables teams to manage their own DNS without touching the parent zone

The Root Name Servers

- There are **13 logical root servers** (named A through M)
- Each is **mirrored globally** using anycast routing — hundreds of physical locations
- All mirrors of a root server share the **same IP address**
- Requests route to the nearest mirror automatically
- Root server operations run by volunteer organizations (universities, ICANN, Verisign, etc.)

DNS Resource Record Types

What is a Resource Record?

Every piece of DNS data is stored as a **resource record (RR)**. Each record has the same general format:

- **NAME:** The domain this record applies to
- **TTL:** How long resolvers can cache this record (in seconds)
- **CLASS:** Almost always IN (Internet)
- **TYPE:** What kind of record this is
- **VALUE:** The data — varies by record type

NAME	TTL	CLASS	TYPE	VALUE
example.com	3600	IN	A	93.184.216.34
www	3600	IN	CNAME	example.com.
example.com	3600	IN	MX	10 mail.example.com.

The SOA Record (Start of Authority)

Every zone must have exactly **one SOA record**. It is the first record in a zone file.

```
domain.com.   IN   SOA   ns1.domain.com.   admin.domain.com. (
                         2024031201    ; serial number (YYYYMMDDNN)
                         3h            ; refresh interval
                         30m           ; retry interval
                         3w            ; expiry period
                         1h            ; negative TTL (NXDOMAIN cache
time)
                        )
```

Field	Meaning
Serial Number	Increments on every change; secondaries sync when this increases
Refresh	How often secondaries check for updates
Retry	How long to wait before retrying a failed refresh
Expiry	How long a secondary will serve data if it cannot reach the primary
Negative TTL	How long to cache "this domain does not exist" responses

A and AAAA Records

- **A** = "address" — IPv4 (32-bit, e.g., 93.184.216.34)
- **AAAA** = IPv6 (128-bit, e.g., 2001:db8::1) — four times the bits of A, hence four A's
- In cloud: when you deploy an EC2 instance or GCE VM, you create an A record pointing to its public IP

CNAME Record (Canonical Name / Alias)

- CNAME always points to **another domain name**, never directly to an IP
- The resolver follows the chain until it reaches an A or AAAA record
- **Important restriction:** A CNAME **cannot coexist** with other records at the same name
  - You cannot put a CNAME on a bare/apex domain (e.g., example.com) — only on subdomains
  - **Cloud workaround:** Route 53 has **ALIAS records** and GCP Cloud DNS has synthetic records that behave like CNAMEs but work at the apex

MX Record (Mail Exchange)

- Directs email delivery for a domain
- The number is the **priority** — lower wins (10 is tried before 20)
- If the primary mail server (priority 10) fails, email falls back to priority 20
- Used with Google Workspace, Microsoft 365, and custom mail servers

NS Record (Name Server)

- These are the records that tell the Internet **which servers to ask** about example.com
- When you register a domain and point it to Route 53 or Cloud DNS, you are updating NS records
- Always have **at least two** NS records for redundancy

TXT Record

- Originally designed for free-form text; now used for many verification purposes
- **SPF** — email sender policy (which servers are allowed to send email for your domain)
- **DKIM** — email signing keys (stored as TXT records, used to verify email authenticity)
- **Domain ownership verification** — Google, AWS ACM, GitHub, etc. ask you to add a TXT record to prove you own the domain

CAA Record (Certification Authority Authorization)

- Controls which Certificate Authorities can issue SSL/TLS certificates for your domain
- If no CAA record exists, any CA can issue certificates
- Inherited by subdomains
- **Cloud relevance:** When using AWS ACM or Google-managed certificates, you may need to add a CAA record if you have strict CA policies

PTR Record (Reverse DNS)

- **Reverse DNS** maps an IP address back to a hostname
- Used by email servers, security tools, and network diagnostics
- Stored under the special in-addr.arpa domain (IPv4) or ip6.arpa (IPv6)
- **Cloud tip:** For EC2 instances running mail servers, AWS allows you to request reverse DNS delegation for your Elastic IPs

TTL Strategies in Cloud Environments

Scenario	Recommended TTL	Reason
Stable production service	300–3600s	Fewer lookups, better cache performance
Before a planned migration	60s	Changes propagate in under a minute
Active migration in progress	60s	Allows quick rollback
After migration is stable	300–3600s	Restore normal caching
CDN / load balancer	60–300s	IPs may change, keep TTL moderate

Name Resolution: Recursive vs. Iterative

Two Query Modes

- RECURSIVE QUERY
  - Client asks resolver: "Give me the answer for www.example.com"
  - Resolver does ALL the work (root → TLD → authoritative)
  - Resolver returns the final answer to the client
  - Client only makes ONE query

- ITERATIVE QUERY
  - Client asks server: "Do you know www.example.com?"
  - Server responds: "No, but try asking this TLD server"
  - Client asks TLD: "Do you know www.example.com?"
  - TLD responds: "No, but try asking example.com's NS"
  - Client asks example.com NS: "Do you know www.example.com?"
  - NS responds: "Yes! The IP is 93.184.216.34"
  - Client makes MULTIPLE queries

- In practice: **your browser makes a recursive query** to your local resolver (ISP or 8.8.8.8). The resolver then uses **iterative queries** to walk down the DNS tree on your behalf.

The 8 Steps of a Full DNS Lookup

- Browser asks recursive resolver: "What is www.example.com?"
- Resolver asks root server: "Who handles .com?"
- Root returns the .com TLD server address
- Resolver asks .com TLD: "Who handles example.com?"
- TLD returns example.com's authoritative name server address
- Resolver asks example.com's NS: "What is the IP for www.example.com?"
- Authoritative NS returns the A record: 93.184.216.34
- Resolver returns the answer to the browser (and caches it for TTL duration)
- Browser connects to 93.184.216.34 via HTTP/HTTPS
- Web server sends back the page

```
Browser ──1──► Recursive Resolver
                    │
                  ──2──► Root Server (.)
                    │◄──3── "Ask the .com TLD server"
                    │
                  ──4──► .com TLD Server
                    │◄──5── "Ask example.com's nameserver"
                    │
                  ──6──► example.com Authoritative NS
                    │◄──7── "www.example.com = 93.184.216.34"
                    │
Browser ◄──8─── Resolver returns IP to browser

Browser ──9──► HTTP GET to 93.184.216.34
Browser ◄──10── Web server returns HTML page
```

Why Cloud-Managed DNS?

Self-managing BIND on a server is possible but introduces operational burden. Cloud DNS services are:

- **Managed:** No server patching, software updates, or capacity planning
- **Globally distributed:** Anycast routing — queries go to the nearest PoP automatically
- **Highly available:** SLAs of 99.99%+ (Route 53 commits to 100% SLA)
- **Integrated:** Deep integration with other cloud services (load balancers, ACM, CloudFront, etc.)

Public vs. Private Hosted Zones

	Public Hosted Zone	Private Hosted Zone
Accessible from	Internet	Inside a VPC only
Use case	myapp.com → public load balancer	db.internal → RDS instance
Cost	Per zone + per query	Per zone + per query

Troubleshooting

Troubleshooting DNS: Essential Commands

```
# dig — the primary DNS debugging tool
dig example.com                    # A record lookup
dig example.com MX                 # Query specific record type
dig @8.8.8.8 example.com           # Query a specific resolver
dig +trace example.com             # Trace full resolution chain (root → answer)
dig -x 93.184.216.34               # Reverse DNS lookup (PTR)

# nslookup — simpler alternative
nslookup example.com               # Basic lookup
nslookup -type=MX example.com      # MX records
nslookup example.com 8.8.8.8       # Use specific resolver

# host — quick lookups
host example.com                   # A record
host -t MX example.com             # MX records
```

Common DNS troubleshooting scenarios

Symptom	Likely Cause	Check
Domain not resolving	NS records wrong at registrar	dig NS yourdomain.com
Change not propagating	TTL too high, cached answer	dig @8.8.8.8 yourdomain.com to bypass local cache
Email not delivering	MX records missing or wrong	dig MX yourdomain.com
HTTPS cert validation fails	TXT/CNAME for domain verification missing	Check registrar DNS
Internal service not resolving	Private zone not associated with VPC	Check Route 53 or Cloud DNS private zone config

Summary

Key Takeaways

- DNS is a globally distributed, hierarchical database — not a single server. It is foundational to every cloud application.
- The DNS namespace is an inverted tree. Domain names are read right-to-left (most general to most specific).
- Zone delegation transfers authority over a subdomain to a new set of name servers. Glue records prevent circular dependencies.
- Record types serve different purposes: A/AAAA for addressing, CNAME for aliases, MX for mail, NS for delegation, TXT for verification, SOA for zone metadata.
- TTL controls how long resolvers cache answers. Lower TTL before changes; raise it again after stability is confirmed.
- Cloud DNS (Route 53, Google Cloud DNS) adds capabilities beyond standard DNS: health checks, routing policies, private zones, and ALIAS records.
- DNS is a common attack vector. Real-world outages (Dyn 2016) show that DNS failures take down entire segments of the Internet.
- When something is broken on the Internet, it is almost always DNS.

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS